

C++ Programming Textbook Notes

Module: Constructors & Destructors in Inheritance | Execution Order & Member Initialization

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. OBJECT LIFECYCLE IN INHERITANCE: CONCEPTUAL FOUNDATIONS

The Problem: Constructing Compound Objects in RAM

In Object-Oriented Programming, a derived class inherits the properties (data members) and behaviors (member functions) of its base class. A derived class object is not merely a single entity—it is a **composite structure** containing both the base class sub-object and the derived class's own members.

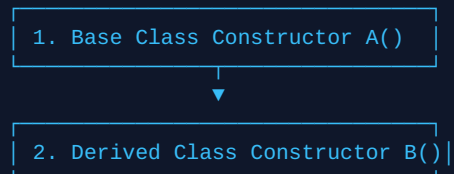
Because the derived class often relies upon or interacts with inherited base data members, a strict construction contract is required: **the foundation (base class) must be fully built and initialized before the upper structure (derived class) can be assembled.**

The Core Execution Rules

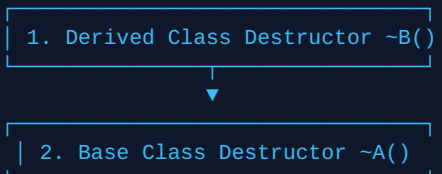
- **Base Object Creation:** When an independent base class object is created, only the constructor and destructor of that base class are invoked.
- **Derived Object Creation (Constructor Order):** When an object of a derived class is created, the **base class constructor executes first**, followed immediately by the **derived class constructor**.
- **Derived Object Destruction (Destructor Order):** When a derived class object goes out of scope, the process is inverted: the **derived class destructor executes first**, followed by the **base class destructor**.

[LIFECYCLE ORDER IN CLASS HIERARCHIES]

CONSTRUCTION ORDER (Top-Down):



DESTRUCTION ORDER (Bottom-Up / LIFO):



2. DEFAULT CONSTRUCTORS IN INHERITANCE

Implicit Base Constructor Invocation

If the base class has a **default constructor** (a constructor with no parameters), the derived class constructor does not need to explicitly call it. The C++ compiler automatically generates and inserts the base constructor call into the derived class constructor's initialization list:

[COMPILER SYNTHESIS OF DEFAULT BASE CONSTRUCTOR]

Programmer Writes in Derived: <pre>B() { // Derived code }</pre>	Compiler Rewrites Internally: <pre>B() : A() { // Implicitly calls A() first! // Derived code }</pre>
---	--

```
// Default Constructor Execution Flow in Inheritance
#include <iostream>
using namespace std;

class A {
public:
    A() {
        cout << "Hello from Base Class A Constructor" << endl;
    }
};

class B : public A {
public:
    // Automatically calls A() before executing its own body
    B() {
        cout << "Hello from Derived Class B Constructor" << endl;
    }
};

int main() {
    B obj;
    // Output:
    // 1. Hello from Base Class A Constructor
    // 2. Hello from Derived Class B Constructor
    return 0;
}
```

3. PARAMETERIZED CONSTRUCTORS: MEMBER INITIALIZATION LIST

The Problem: Base Class Lacks a Default Constructor

If the base class defines a **parameterized constructor** and does *not* define a default constructor, the compiler cannot guess what values to initialize the base sub-object with! In this case, attempting to write `B() {}` triggers a fatal compile-time error.

The Solution: Explicit Call via Member Initialization List

The derived class constructor must **explicitly pass arguments to the base class constructor** using a **Member Initialization List** syntax:

```
DerivedClass(type1 arg1, type2 arg2) : BaseClass(arg1) {
    // Derived class initialization using arg2
}
```

Complete Parameterized Inheritance Program

```
// Passing Arguments from Derived Constructor to Base Constructor
#include <iostream>
using namespace std;

class A {
private:
    int a;

public:
    // Parameterized Constructor for Base Class
    A(int k) {
        a = k;
        cout << "Base Class A initialized with a = " << a << endl;
    }
};

class B : public A {
private:
    int b;

public:
    // Derived constructor receives two arguments: passes 'y' to Base A(y)
    B(int x, int y) : A(y) {
        b = x;
        cout << "Derived Class B initialized with b = " << b << endl;
    }
};

int main() {
    // Instantiating B with arguments (5, 7)
    // 7 is forwarded to A(7), 5 is assigned to b
    B obj(5, 7);
    return 0;
}
```

4. DESTRUCTORS IN INHERITANCE: THE LIFO TEARDOWN

Conceptual Explanation

While constructors build an object from the inside out (Base first, then Derived), **destructors tear it down from the outside in**.

The derived class destructor executes its body *first* to release any derived-level resources (heap pointers, open files). Once the derived destructor finishes, the compiler **automatically injects a call to the base class destructor**. The base destructor executes *second* to clean up base-level resources.

```
// Demonstrating Destructor Order in Inheritance
#include <iostream>
using namespace std;

class A {
public:
    A() { cout << "Constructor A" << endl; }
    ~A() { cout << "Destructor ~A (Base cleans up second)" << endl; }
};

class B : public A {
public:
    B() { cout << "Constructor B" << endl; }
    ~B() {
        cout << "Destructor ~B (Derived cleans up first)" << endl;
        // Compiler automatically calls ~A() right after this block!
    }
};

int main() {
    {
        B obj;
        // Constructed: Constructor A -> Constructor B
    } // Block scope exits here!
    // Destroyed: Destructor ~B -> Destructor ~A
    return 0;
}
```

5. CONSTRUCTOR & DESTRUCTOR EXECUTION ORDER RULES

Inheritance Type	Constructor Order (Construction)	Destructor Order (Destruction)
Single Inheritance (class B : public A)	A() → B()	~B() → ~A()
Multilevel Inheritance (class C : public B → B : public A)	A() → B() → C()	~C() → ~B() → ~A()
Multiple Inheritance (class C : public A, public B)	A() → B() → C() (Order of declaration in class header!)	~C() → ~B() → ~A() (Exact reverse order of declaration)

Important Rule: Declaration Order Governs Multiple Inheritance
In multiple inheritance (e.g., class C : public A, public B), the base constructors are called in the order they are **declared in the class header** (A first, then B), *not* the order they appear in the constructor's initialization list!

Senior Developer Best Practice: Virtual Destructors
When designing polymorphic base classes meant to be manipulated via base class pointers (e.g., Base* ptr = new Derived(); delete ptr;), always declare the base class destructor as **virtual** (virtual ~Base() {}). Without a virtual destructor, delete ptr invokes only the base destructor, causing a severe resource leak by skipping the derived destructor!